

Getting started: how to start the project, step by step

Follow these steps in order. Each step tells you what success looks like before you move on. The full reference, with troubleshooting, is [README.md](#).

Step	What you do	Board needed?	Time
1	Put the files on your PC	no	5 min
2	Install the tools (once)	no	1–3 h (large download)
3	Simulate: prove the Verilog works	no	15 min
4	Way 1: FPGA only, type <code>t</code> and see the temperature	yes	30–45 min
5	Way 2: ARM + FPGA with your C program	yes	1 h
6	Verify and collect results for the report	yes	as needed

Step 1: Put the files on your PC

1. Get the project, in either of these ways:
 - Download `smart_serial_controller.zip` (the file you were sent).
 - Or download it from GitHub: <https://github.com/Venkatesh5454/Active-noise-cancellation-for-defence-system/archive/refs/heads/claude/wizardly-cori-h3t0hy.zip>. You can also open the repository on GitHub, change the branch drop-down from **main** to **claude/wizardly-cori-h3t0hy**, and click **Code** → **Download ZIP**. The `main` branch does **not** contain this project.
2. On Windows, right-click the ZIP and choose **Properties**. If there is an **Unblock** box, tick it and click **OK**.
3. Right-click the ZIP and choose **Extract All...**. Set the destination to `C:\ssc`, a short path with no spaces, and click **Extract**.
4. Check that `C:\ssc\smart_serial_controller\README.md` exists.
 - If you have `C:\ssc\smart_serial_controller\smart_serial_controller\README.md` instead, move the inner `smart_serial_controller` folder up one level.
 - For the GitHub ZIP, copy the `smart_serial_controller` folder from inside the extracted `Active-noise-...` folder to `C:\ssc\`.

Do not work inside Downloads, Desktop or OneDrive. Vivado has problems with spaces in folder names and with paths longer than 260 characters.

What the folders contain:

```

smart_serial_controller\
  GETTING_STARTED.md    <- this file
  README.md             <- full guide + troubleshooting (section 7)
  windows\              <- double-click scripts: 1_simulate, 2_build_way1, 3_build_way2, 4_open_gui
  rtl\                  <- the Verilog design (13 files)
  boards\zedboard\      <- ZedBoard top levels and pin constraints (.xdc)
  sim\                  <- testbenches and fake devices
  vivado\               <- Tcl build scripts (the .bat files run these)
  sw\                   <- C program for the ARM (Vitis)
  docs\                 <- code explanation, register map, expected simulation output
  docs\pdf\             <- the same guides as PDF files, easy to read or print

```

Step 2: Install the tools (once)

1. **Vivado and Vitis.** Download the *AMD Unified Installer* from the AMD website, which needs a free AMD account. Choose the product **Vitis**, which installs both Vivado and Vitis.
 - Any version from 2020.2 onwards works; the current release is fine.
 - Install to the default folder, and write down the folder the installer shows. You need it if a .bat file says "Could not find Vivado". It looks like:
 - C:\AMDDesignTools\<version>\Vivado or C:\Xilinx\<version>\Vivado for 2025.1 and newer
 - C:\Xilinx\Vivado\<version> for older versions
 - Under devices you only need **SoCs → Zynq-7000**. Untick the others to save disk space.
 - Make sure **Install Cable Drivers** is ticked.
 - The free licence covers the ZedBoard's XC7Z020, so no licence file is needed.
2. **ZedBoard board files.** This is only needed for Step 5, but do it now. Start Vivado, then go to **Tools → Vivado Store... → Boards**, click **Refresh**, open **Avnet**, then **ZedBoard → Install**.
 - In Vivado 2020.x, use instead **File → New Project → ... → Boards** tab → **Refresh**, then the download icon next to ZedBoard.
3. **A serial terminal.** Install Tera Term or PuTTY.
4. **Windows drivers.** You usually need nothing extra. Fix these only if they come up later:
 - If the ZedBoard's J14 port shows no COM port in Device Manager, install the Cypress CY7C64225 USB-UART driver.
 - If Hardware Manager → Auto Connect finds no board, re-install the cable driver as described in README section 2.
5. **Linux only:** run the cable-driver installer once as root:


```
<install>/<version>/data/xicom/cable_drivers/lin64/install_script/install_drivers/install_drivers
```

 (for versions before 2025.1, use `<install>/Vivado/<version>/data/...`)

Step 3: Simulate, with no board needed

This proves the Verilog works before you touch any hardware.

Windows: double-click `windows\1_simulate.bat`.

If Windows shows "Windows protected your PC", click *More info → Run anyway*. If the script says "Could not find Vivado", open `windows\_find_vivado.bat` in Notepad and set `MY_VIVADO_SETTINGS` to your `settings64.bat`. Write it with one pair of quotes around the whole line, for example `set "MY_VIVADO_SETTINGS=C:\Xilinx\2025.1\Vivado\settings64.bat"`. The path is `<root>\<version>\Vivado\settings64.bat` for 2025.1 and newer, and `C:\Xilinx\Vivado\<version>\settings64.bat` for older versions.

Linux:

```
cd ~/ssc/smart_serial_controller
source <Vivado install folder>/settings64.sh
vivado -mode batch -source vivado/run_sim.tcl -tclargs tb_ssc_top
vivado -mode batch -source vivado/run_sim.tcl -tclargs tb_ssc_axi
vivado -mode batch -source vivado/run_sim.tcl -tclargs tb_zed_standalone
```

Success. Each testbench prints its own report, among many Vivado INFO lines (those are normal). Each report ends with a box:

```
=====
ALL 411 CHECKS PASSED
=====
```

The box says 411 for tb_ssc_top, 9 for tb_ssc_axi and 10 for tb_zed_standalone. On Windows, the final **SUMMARY** shows `passed` for all three.

- `m OF n CHECKS FAILED` would mean a real problem.
- The Vivado logs are `build\sim_tb_*.log`.
- The reference logs in `docs\sim_results\` were made with a different simulator (Icarus Verilog), so only the lines printed by the testbench should match: `[1] ... , ok ...` and the box.

To see waveforms (for your report screenshots):

1. Run `windows\2_build_way1_fpga_only.bat` once. It also creates the project. Then run `windows\4_open_project_gui.bat`.
2. In *Sources* → *Simulation Sources* → *sim_1*, right-click `tb_ssc_top` and choose **Set as Top**.
3. Click **Run Simulation** → **Run Behavioral Simulation**.
4. In the Tcl Console at the bottom, type `log_wave -r /` and press Enter, then type `run all`. Without `log_wave`, signals you drag in later are empty.
5. In the **Scope** panel, expand `tb_ssc_top` → `dut` → `u_i2c`, then drag `st` from the **Objects** panel into the waveform window. You can also type:

```
add_wave /tb_ssc_top/scl /tb_ssc_top/sda /tb_ssc_top/dut/u_i2c/st
```

Step 4: Way 1, FPGA only

4a. Build the bitstream

Double-click `windows\2_build_way1_fpga_only.bat`. It takes 5-15 minutes; do not close the window.

Success: you see the box `Way 1 build finished`, with `worst setup slack (WNS)` at 0 or more. After it comes `SUCCESS`. Bitstream: `build\standalone\zed_top_standalone.bit`.

If it says `*** BUILD FAILED ***`, open `build\build_way1.log` and search for `ERROR`.

4b. Prepare the board (power OFF)

1. **Boot-mode jumpers:** set JP7, JP8, JP9, JP10 and JP11 all to the **GND** side. This is JTAG mode.
2. **Plug in the Pmods.** Pin 1 of each Pmod goes to pin 1 of the ZedBoard connector; it is marked on both.
 - **JA:** PmodUSBUART, top row. Set its power jumper **JP1 to LCL**.

- **JB:** PmodSF3, using the whole 12-pin connector.
 - **JC:** PmodTMP2. Put its 2×4 header in the four columns of JC **nearest the GND/VCC end**: pins 3–6 on the top row and 9–12 on the bottom row. This puts SCL on JC3 and SDA on JC4. JC1, JC2, JC7 and JC8 stay empty. Leave both address jumpers on the PmodTMP2 **open**, which gives address 0x4B.
 - **I2C pull-ups:** try without them first, because the FPGA's internal pull-ups usually work. If `t` later says *No ACK*, add two 4.7 kΩ resistors, SCL to 3.3 V and SDA to 3.3 V, on a breadboard. README section 2 (Hardware) explains how to wire them.
3. Connect the cables:
 - **J17 (PROG)** micro-USB to the PC.
 - The **PmodUSBUART** micro-USB to the PC.
 - The 12 V supply.
 4. Switch the board on. The green POWER LED lights.

4c. Program the FPGA

1. Open Vivado (the GUI).
2. Click **Open Hardware Manager → Open Target → Auto Connect**.
3. Click **Program Device**, browse to `build\standalone\zed_top_standalone.bit`, and click **Program**.
4. The blue **DONE** LED lights, and LD0 starts blinking.

4d. Talk to it

1. Open **Device Manager → Ports (COM & LPT)**. The PmodUSBUART appears as **USB Serial Port (COMx)**. If you are not sure which port it is, unplug the cable and see which COM port disappears.
2. In Tera Term or PuTTY, open that COM port at **115200 baud, 8 data bits, no parity, 1 stop bit, no flow control**.
3. Press **BTNC** (the centre push button) on the ZedBoard.

Success:

```
Smart Serial Controller - Way 1 bring-up
t = temperature (PmodTMP2 on JC)
f = flash JEDEC ID (PmodSF3 on JB)
> t
Temp = +24.6875 C
> f
Flash ID = 20 BA 19
```

Touch the sensor with your finger and press `t` again: the temperature should rise. If anything is wrong, see README section 7.

Step 5: Way 2, ARM + FPGA (the final system)

5a. Build the hardware

Double-click `windows\3_build_way2_arm_fpga.bat`. It takes 10–20 minutes.

Success: you see the box `Way 2 build finished`, with WNS at 0 or more. After it comes `SUCCESS`.
Hardware for Vitis: `build\ps_system\ssc_system.xsa`.

If it says `*** BUILD FAILED ***`, open `build\build_way2.log` and search for `ERROR:`. The usual cause is missing ZedBoard board files (see Step 2).

To see the block design (the architecture slide, drawn by Vivado), open `build\ps_system\ssc_ps.xpr` in Vivado and choose **Open Block Design**.

5b. Create the software in Vitis

Vitis 2023.2 or newer (Unified IDE):

1. Start Vitis and choose an empty workspace folder, for example `C:\ssc\vitis_ws`.
2. Go to **File → New Component → Platform**:
 - Name it `ssc_platform`.
 - For the hardware design, browse to `build\ps_system\ssc_system.xsa`.
 - Choose standalone, ps7_cortexa9_0, then **Finish**.
 - Click **Build** in the Flow panel.
3. Go to **File → New Component → Application**:
 - Name it `ssc_app`, select `ssc_platform`, then **Finish**.
4. Copy the four files from `sw\` (`main.c`, `ssc_driver.c`, `ssc_driver.h`, `ssc_regs.h`) into `ssc_app\src`.
5. Click **Build**.

Vitis 2020.2 - 2023.1 (Classic):

1. Go to **File → New → Application Project → Next**.
2. Choose **Create a new platform from hardware (XSA)** and browse to the `.xsa` file.
3. Name the project `ssc_app` and select processor `ps7_cortexa9_0`.
4. Choose the template **Empty Application (C)**, then **Finish**.
5. Right-click `ssc_app/src` and choose **Import Sources**. Pick the `sw` folder and all four files.
6. Click the hammer icon to build.

5c. Run

1. Also connect the **J14 (UART)** micro-USB. This is the ARM's console, and it appears as another COM port.
2. Open a terminal on that COM port at **115200 8N1**.
3. Start the program:
 - **Unified IDE:** click **Run** in the Flow panel.
 - **Classic:** right-click `ssc_app` and choose **Run As → Launch Hardware**.

Vitis loads the bitstream and starts your program.

Success:

```
=== Smart Serial Controller - Way 2 (ARM + FPGA) ===
reading the controller ID at 0x43c00000 (a hang here = bitstream not loaded)
controller found at 0x43c00000
...
ssc> selftest      -> four PASS lines (works even with no Pmods)
ssc> temp          -> +24.xxxx C
ssc> id            -> 20 ba 19
ssc> flash         -> read back 256 bytes: 0 errors PASS
ssc> bench         -> CPU time with and without the FIFO
```

The full expected output is in README section 5.3.

Step 6: Verify and collect results

Use the checklist in **README section 6**. It has four levels:

1. **Simulation:** the PASSED messages, plus waveform screenshots.

2. **Way 1 on the board.**
3. **Way 2 on the board.**
4. **Logic analyser on JD:** the I2C capture must match slide 13.

For the report, collect these numbers:

Number	Where it comes from
Utilisation	build\standalone\utilization.rpt
Timing (WNS)	timing_summary.rpt
Baud error table	test [9] in the simulation log
CPU time saved	the bench command
Bytes moved by the bridge	bridge off

How to learn the code (suggested reading order)

1. [docs/CODE_WALKTHROUGH.md](#), the explanation of every file.
2. The RTL, in this order (each file starts with a comment block):
 1. ssc_sync_filter.v
 2. ssc_fifo.v
 3. ssc_baud_gen.v
 4. ssc_uart.v
 5. ssc_spi.v
 6. ssc_i2c.v
 7. ssc_bridge.v
 8. ssc_irq.v
 9. ssc_regbank.v
 10. ssc_apb_top.v
 11. ssc_axi_apb_bridge.v
 12. ssc_axi_top.v
 13. ssc_cmd_fsm.v
3. [docs/REGISTER_MAP.md](#) before you change the C code.
4. sim/tb_ssc_top.v: each test task t_... shows how to use one feature from software.

How this fits your 14-week plan

Weeks	Slide stage	In this folder
1-3	Spec + RTL	rtl\ (done; study and explain it)
3-7	Simulation	sim\ (Step 3)
7-9	Synthesis and timing	Step 4a / 5a reports
9-12	ZedBoard bring-up	Steps 4-5
12-14	Results and report	Step 6